

Projeto Prático: Otimização e Cálculo Numérico de Autovalores

Disciplina: Equações Diferenciais Ordinárias e Álgebra Linear Computacional

Semestre letivo corrente

1 Introdução

O cálculo de autovalores e autovetores é fundamental para a resolução de sistemas de Equações Diferenciais Ordinárias (EDOs), análise de estabilidade e compressão de dados. Pelo Teorema de Abel-Ruffini, não existe fórmula fechada para encontrar as raízes do polinômio característico de matrizes de ordem $N \geq 5$. Portanto, a computação moderna recorre a métodos numéricos iterativos estruturados.

Objetivo do Projeto: Desenvolver um algoritmo capaz de calcular os autovalores de matrizes quadradas genéricas, realizar uma análise de escalabilidade (consumo de memória e tempo) e implementar otimizações baseadas nas propriedades estruturais da matriz.

2 Fase 1: Implementação do Algoritmo Geral

Os grupos deverão implementar, na linguagem de programação de sua escolha (Python, C++, Java, Julia, etc.), a esteira clássica para o cálculo de autovalores:

- 1.1 Redução à Forma de Hessenberg:** Implementar um algoritmo (recomenda-se o uso de Transformações de Householder) para transformar uma matriz densa genérica A em uma matriz na forma de Hessenberg Superior H .
- 1.2 Iteração QR:** Implementar o algoritmo QR iterativo. O algoritmo deve receber a matriz de Hessenberg H e aplicar a fatoração $H = QR$ iterativamente ($H_{k+1} = R_k Q_k$) até que os elementos da subdiagonal converjam para zero (dada uma tolerância ϵ , ex: 10^{-10}).

3 Fase 2: Estudo de Escalabilidade (Tempo e Memória)

Com o algoritmo base funcionando, o grupo deverá realizar testes empíricos de complexidade computacional.

- 2.1** Gerar matrizes aleatórias quadradas de tamanhos crescentes: $N \in \{10, 50, 100, 250, 500, 1000\}$.
- 2.2** Medir o **tempo de execução** para encontrar todos os autovalores de cada matriz.
- 2.3** Medir o **pico de consumo de memória RAM** durante a execução do algoritmo para cada N .

- 2.4 Construir gráficos de dispersão: $N \times \text{Tempo}$ e $N \times \text{Memória}$. Discutir qual parece ser a complexidade empírica ($O(N^2)$, $O(N^3)$, etc.).

4 Fase 3: Otimização por Restrição Estrutural

A verdadeira engenharia de software científico brilha ao explorarmos restrições matemáticas para otimizar o código.

- 3.1 **Matrizes Simétricas:** Adapte ou crie uma variação do seu código para lidar exclusivamente com matrizes simétricas ($A = A^T$).
- 3.2 **Redução Tridiagonal:** Demonstre empiricamente (e imprima no console para $N = 5$) que ao aplicar as reflexões de Householder em uma matriz simétrica, ela não apenas adota a forma de Hessenberg, mas colapsa para uma **Matriz Tridiagonal Simétrica**.
- 3.3 **Novo Estudo de Desempenho:** Repita o estudo de escalabilidade da Fase 2, mas agora gerando apenas matrizes simétricas aleatórias e utilizando a estrutura tridiagonal.
- 3.4 Sobreponha os resultados (Tempo e Memória) do algoritmo geral com o otimizado em um mesmo gráfico e quantifique o ganho percentual de desempenho.

5 Fase 4: Aplicação Prática – Dinâmica de Redes de Reatores

Contexto (Engenharia Química): Considere uma planta industrial com uma rede de N reatores de mistura perfeita (CSTRs) conectados em série, onde ocorre uma reação química de degradação de primeira ordem. Devido a efeitos de reciclo e difusão, o reator i troca massa tanto com o reator $i - 1$ quanto com o $i + 1$.

A modelagem de massa desse sistema gera um sistema linear de EDOs da forma:

$$\frac{d\mathbf{C}}{dt} = A\mathbf{C} \quad (1)$$

Onde $\mathbf{C}(t)$ é o vetor de concentrações de tamanho N , e A é a matriz de coeficientes ($N \times N$) contendo as taxas de vazão, difusão e a constante cinética da reação. Pela natureza do problema (conexões apenas com vizinhos imediatos), a matriz A resultante é uma **matriz tridiagonal**.

Contexto (Ciência da Computação): Para simular esse sistema no tempo, precisamos usar métodos de integração numérica (solvers de EDO). A escolha do algoritmo computacional ideal depende da **Razão de Rigidez (Stiffness Ratio)** do sistema. Se o sistema for muito "rígido" (stiff), métodos explícitos rápidos (como Runge-Kutta 4) falham catastroficamente e exigem passos de tempo infinitesimais, obrigando o uso de métodos implícitos muito mais pesados computacionalmente.

A rigidez de um sistema linear de EDOs é definida pela razão entre o maior e o menor valor absoluto das partes reais de seus autovalores:

$$S = \frac{\max |\operatorname{Re}(\lambda_i)|}{\min |\operatorname{Re}(\lambda_i)|} \quad (2)$$

Sistemas onde $S \gg 1$ (geralmente $S > 10^3$) são classificados como sistemas computacionalmente rígidos (Stiff ODEs).

O Desafio do Grupo:

- 4.1** Construa a matriz tridiagonal A para $N = 100$ reatores. Assuma os seguintes valores na diagonal principal e sub/superdiagonais (representando as taxas adimensionais normalizadas):
- Diagonal principal: -2.5 (perda por escoamento e reação).
 - Superdiagonal e Subdiagonal: 1.0 (troca com vizinhos).
- 4.2** Utilize o **algoritmo otimizado (Fase 3)** desenvolvido por vocês para encontrar todos os autovalores de A .
- 4.3 Análise de Estabilidade:** Verifique se todos os autovalores possuem parte real negativa (condição matemática para que a reação química no sistema atinja um estado estacionário estável, sem explodir).
- 4.4 Análise Computacional:** Calcule a razão de rigidez S usando os autovalores encontrados. Com base nesse valor, discuta no relatório técnico se um engenheiro de software deveria utilizar um solver explícito (para EDOs não-rígidas) ou um solver implícito (para EDOs rígidas) ao programar o simulador dessa planta química.

6 Entregáveis

Cada grupo deverá entregar:

- **Código-Fonte:** Hospedado em um repositório git (GitHub, GitLab, etc.), devidamente comentado e com um arquivo `README.md` explicando como compilar e executar.
- **Relatório Técnico (PDF):** Um documento contendo:
 - Breve explicação matemática dos métodos implementados.
 - Os gráficos gerados nas Fases 2 e 3.
 - Discussão dos resultados: Por que matrizes simétricas tornam o método tão mais rápido? O gargalo do seu programa foi CPU ou Memória?